

Rising Waters

Machine Learning-Based Flood Prediction System

Project Name	Rising Waters — Machine Learning-Based Flood Prediction System
Date	04 July 2026
Author	Lahari Sri Kotipalli, B.Tech – Computer Science and Engineering
Deployment Platform	Render (Gunicorn)

Project Description

Rising Waters is a machine learning-based web application that predicts flood occurrence from historical weather and rainfall parameters. The project covers the full ML lifecycle data preprocessing, model training, evaluation, model serialization, and deployment through a Flask web application.

The system takes ten meteorological and rainfall readings temperature, humidity, cloud cover, annual rainfall, and four seasonal rainfall breakdowns (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), average June rainfall, and subdivision rainfall and classifies whether a flood is likely. The best-performing scikit-learn classifier and its fitted feature scaler are serialized with Joblib (floods.save and transform.save) and loaded once at Flask startup. Results are shown on a dedicated result page with an animated risk gauge, and the whole interface supports a persisted light/dark theme along with an ambient rain-and-lightning canvas animation.

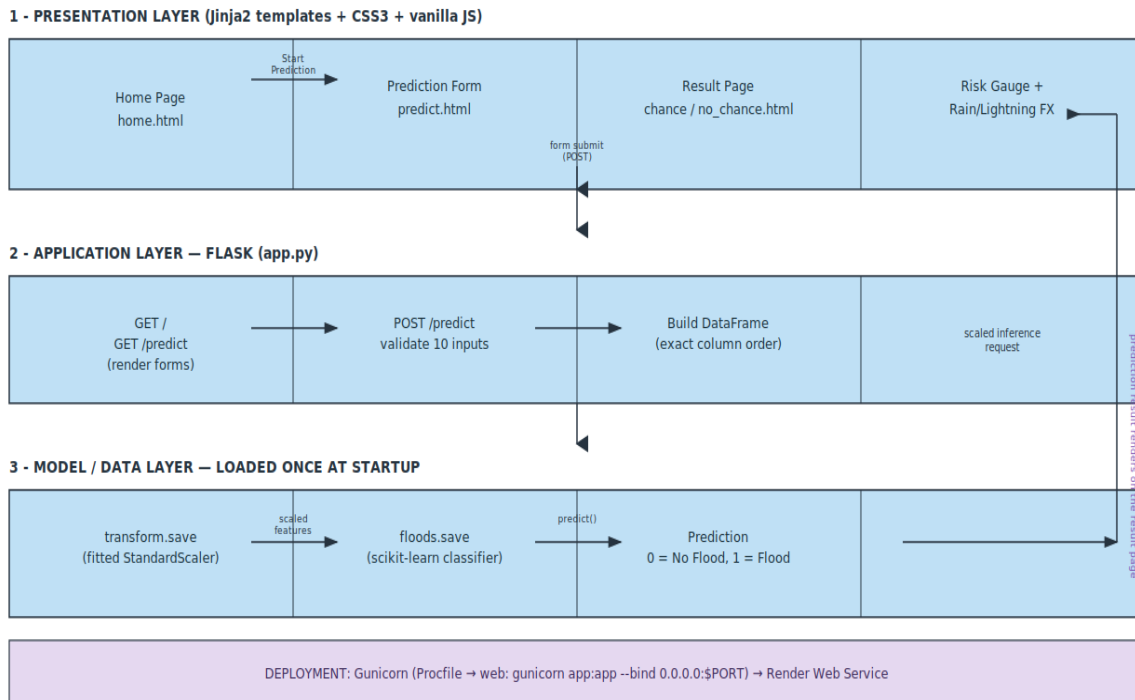
Scenarios

Scenario 1: A resident checks the app during a heavy monsoon week, entering high humidity, high cloud cover, and high seasonal rainfall figures. The system returns a Flood Predicted result, with the risk gauge animating into the high range and prompting early precaution.

Scenario 2: The same resident checks again during a dry spell with low rainfall and humidity values. The system returns a No Flood Predicted result, with the gauge settling in the low range and offering reassurance.

Scenario 3: An evaluator tests the deployed app's /predict endpoint directly with a partial payload (missing fields) using Postman or curl, confirming the app fails safely with a handled validation error instead of crashing.

Technical Architecture



Description: Rising Waters uses a three-layer architecture. The Presentation Layer (Jinja2 templates, custom CSS3 with light/dark theme variables, and vanilla JavaScript) renders the input form, the risk gauge, and the rain/lightning canvas effect. The Application Layer (Flask routes in app.py) handles form validation, assembles the ten readings into a DataFrame with the exact training column order, and dispatches them to the model. The Model / Data Layer is a Joblib-serialized scikit-learn classifier and its fitted scaler, both loaded once at process startup rather than per-request. In production the app runs behind Gunicorn and is deployed to Render.

(Note: this project does not use a database the trained model artifacts, floods.save and transform.save, serve the same role a persistent data store would play in other architectures.)

Pre-requisites

- **Python Programming Proficiency:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Libraries:** scikit-learn, Pandas, and NumPy documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Version Control with Git:** Git Documentation
- **Model Serialization:** Joblib Documentation
- **Deployment Basics:** Gunicorn and Render Documentation

Project Workflow

The project was executed across six milestones, moving from data preparation through model training, backend and frontend development, to public deployment and final testing.

Milestone 1: Data Preparation & Model Selection

- Activity 1.1: Collect and clean the historical rainfall and weather dataset (flood_dataset.xlsx).
- Activity 1.2: Perform Exploratory Data Analysis using Pandas, Matplotlib, and Seaborn to examine feature distributions and correlations.
- Activity 1.3: Handle missing values and detect outliers in the rainfall and weather columns.
- Activity 1.4: Fit a feature scaler and define the train/test split used for model comparison.

Milestone 2: Model Training & Evaluation

- Activity 2.1: Train multiple classification models on the scaled features.
- Activity 2.2: Evaluate each model using Accuracy Score, Confusion Matrix, Precision, Recall, F1-Score, and a full Classification Report.
- Activity 2.3: Select the best-performing model based on the evaluation metrics.
- Activity 2.4: Serialize the selected classifier as floods.save and the fitted scaler as transform.save using Joblib, so the Flask app can load them without depending on the training notebook.

Milestone 3: Flask Backend Development (app.py)

Activity 3.1: Define the Core Routes

- GET / — renders home.html, the landing page with the project introduction.
- GET /predict — renders predict.html, the input form for the 10 weather/rainfall readings.
- POST /predict — validates form data, assembles it into a DataFrame with the exact training column names, scales it, runs inference, and renders the matching result template.

```
@app.route('/')
def home():
    return render_template('home.html')

@app.route('/predict', methods=['GET'])
def predict_form():
    return render_template('predict.html')

@app.route('/predict', methods=['POST'])
```

```
def predict():
    data = [float(request.form[f]) for f in FEATURE_ORDER]
    scaled = scaler.transform([data])
    result = model.predict(scaled)[0]
    template = 'chance.html' if result == 1 else 'no_chance.html'
    return render_template(template, risk=result)
```

Activity 3.2: Load the Model and Scaler at Startup

floods.save and transform.save are loaded once with Joblib when the process starts, avoiding the cost of reloading them on every request.

```
import joblib

model = joblib.load('floods.save')
scaler = joblib.load('transform.save')
```

Activity 3.3: Input Validation

- Verify all 10 fields (Temperature, Humidity, Cloud Cover, Annual/Jan-Feb/Mar-May/Jun-Sep/Oct-Dec Rainfall, Average June Rainfall, Subdivision Rainfall) are present and numeric before scoring, returning a handled error otherwise.

```
lahari@ASUS-VivoBook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
$ curl -X POST http://127.0.0.1:5000/predict \
-d "Temp=30" \
-d "Humidity=80" \
-d "CloudCover=60" \
-d "Annual=1200" \
-d "JanFeb=50" \
-d "MarMay=200" \
-d "JunSep=800" \
-d "OctDec=150" \
-d "AvgJune=120" \
-d "Sub=1100"
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flood Prediction Result</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=SpaceGrotesk:wght@500;700&family=IBM+Plex+Sans:wght@400;500&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/main.css">
</head>
<body data-rain="light" data-lightning="off">

<canvas id="rain-canvas"></canvas>
<div class="cloud-layer">
  <div class="cloud c1"></div>
  <div class="cloud c2"></div>
</div>
<div class="lightning-flash"></div>

<nav class="navbar">
  <div class="nav-brand"><span class="dot"></span>Floods Prediction</div>
  <div class="nav-links">
    <a href="/">Home</a>
    <button class="theme-toggle" id="theme-toggle" type="button" aria-label="Toggle light and dark theme">
      <span class="theme-toggle-icon">🌞</span>
    </button>
  </div>
</nav>

<div class="page">

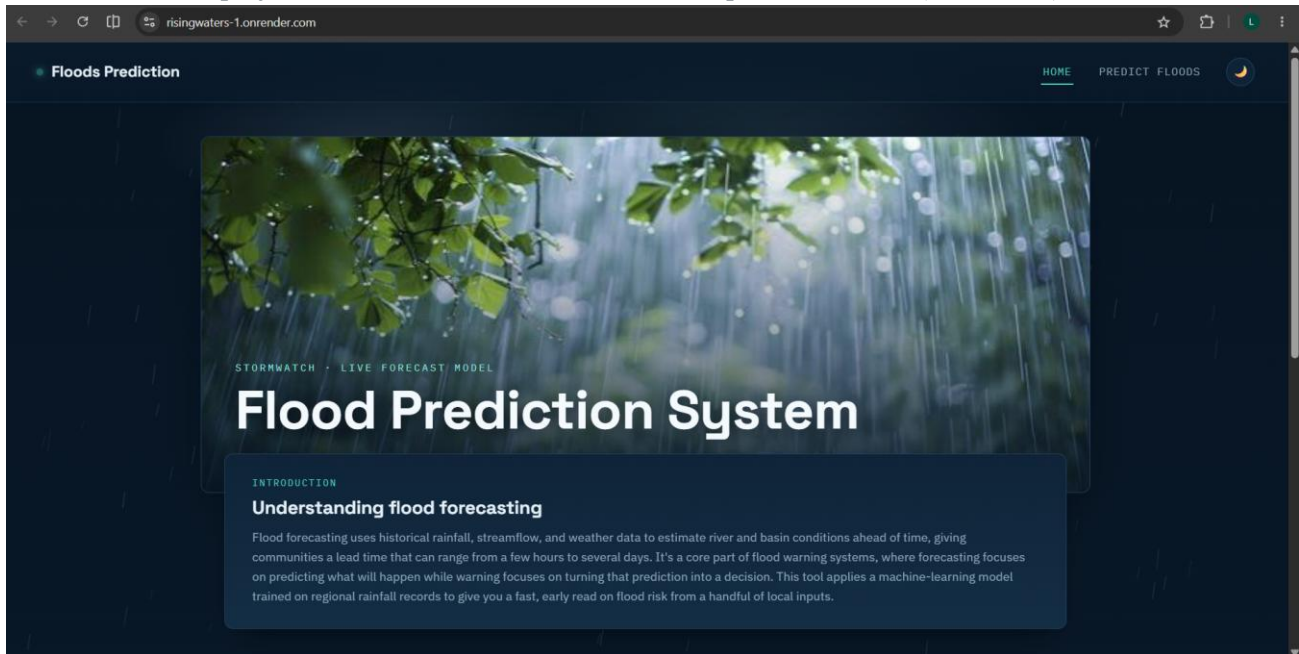
  <div class="brand">
    <span class="dot"></span>
    Station&nbsp;&07 &middledot; Result
  </div>

  <div class="panel">
    lahari@ASUS-VivoBook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
    $ curl -X POST http://127.0.0.1:5000/predict \
    -d "Temp=30" \
    -d "Humidity=80" \
    -d "CloudCover=60" \
    -d "Annual=1200" \
    -d "JanFeb=50" \
    -d "MarMay=200" \
    -d "JunSep=800" \
    -d "OctDec=150" \
    -d "AvgJune=120"
    <!doctype html>
    <html lang=en>
    <title>400 Bad Request</title>
    <h1>Bad Request</h1>
    <p>The browser (or proxy) sent a request that this server could not understand.</p>
    (venv)
```

Milestone 4: Frontend Development

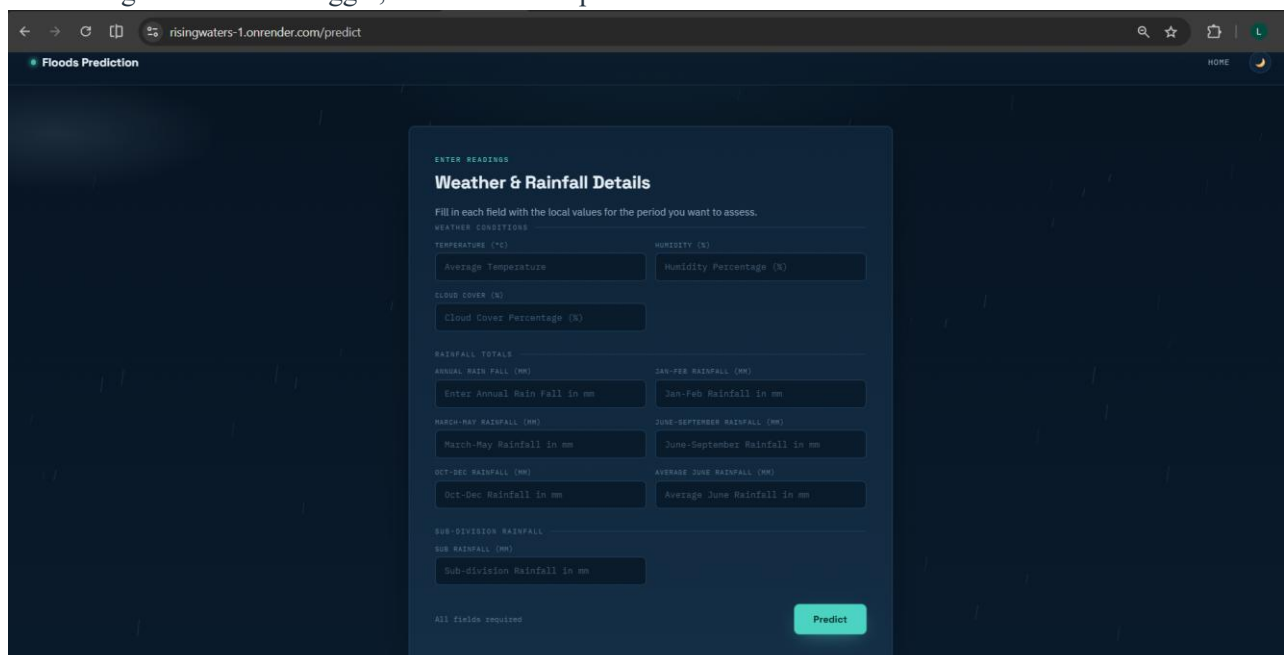
Activity 4.1: Home Page

- Introduces the project with a hero section and links to the prediction form (home.html).



Activity 4.2: Prediction Page

- Build predict.html as a form collecting all 10 model inputs, grouped into Weather Conditions, Rainfall Totals, and Sub-division Rainfall sections.
- Add a light/dark theme toggle, with the choice persisted across visits.



The screenshot shows the prediction page of the Flood Prediction System. The page has a dark blue theme. At the top, there is a navigation bar with the text "Floods Prediction" on the left and "HOME" on the right. Below the navigation bar is a large form titled "Weather & Rainfall Details". The form is divided into three main sections: "Weather Conditions", "Rainfall Totals", and "Sub-division Rainfall".

Weather Conditions

Fill in each field with the local values for the period you want to assess.

WEATHER CONDITIONS

TEMPERATURE (°C)

Average Temperature

HUMIDITY (%)

Humidity Percentage (%)

CLOUD COVER (%)

Cloud Cover Percentage (%)

RAINFALL TOTALS

ANNUAL RAIN FALL (MM)

Enter Annual Rain Fall in mm

JAN-FEB RAINFALL (MM)

Jan-Feb Rainfall in mm

MARCH-MAY RAINFALL (MM)

March-May Rainfall in mm

JUNE-SEPTEMBER RAINFALL (MM)

June-September Rainfall in mm

OCT-DEC RAINFALL (MM)

Oct-Dec Rainfall in mm

AVERAGE JUNE RAINFALL (MM)

Average June Rainfall in mm

SUB-DIVISION RAINFALL

SUB RAINFALL (MM)

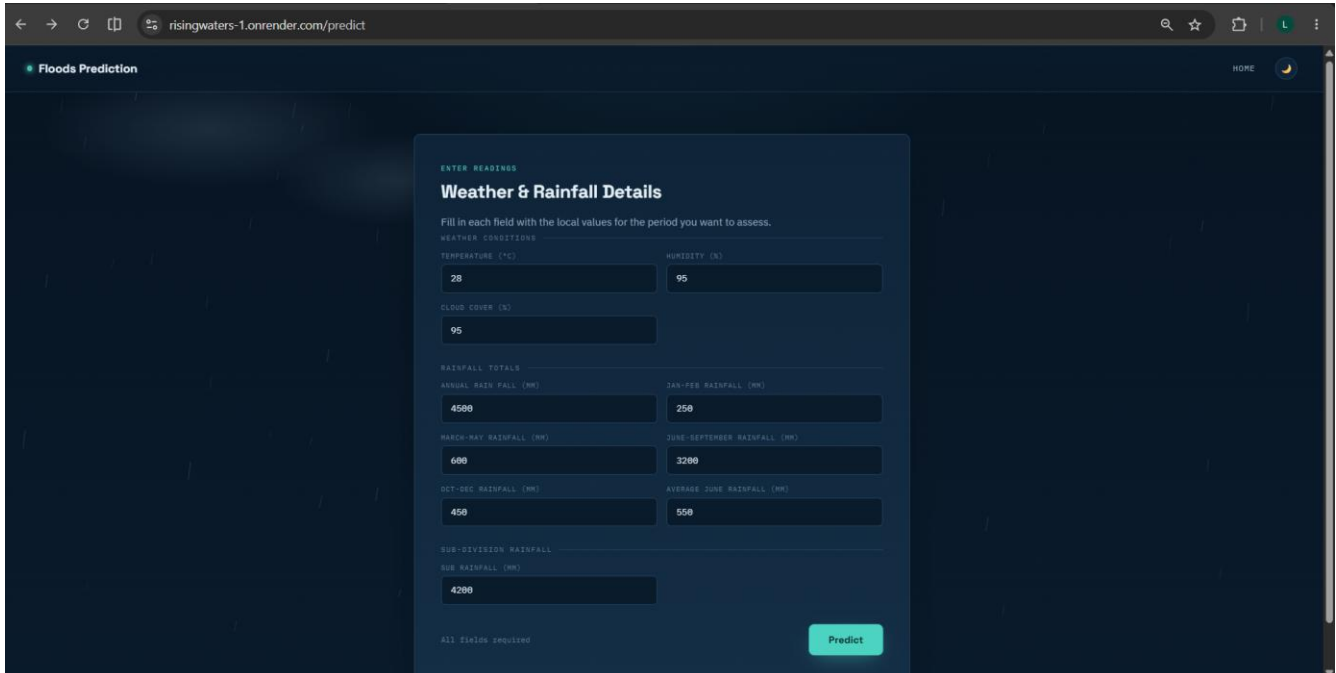
Sub-division Rainfall in mm

All fields required

Predict

Activity 4.3: Result Pages

- `chance.html` — shown when a flood is predicted; displays an animated high-risk gauge.



Floods Prediction

ENTER READINGS

Weather & Rainfall Details

Fill in each field with the local values for the period you want to assess.

WEATHER CONDITIONS

TEMPERATURE (°C) HUMIDITY (%)

CLOUD COVER (%)

RAINFALL TOTALS

ANNUAL RAINFALL (MM) JAN-FEB RAINFALL (MM)

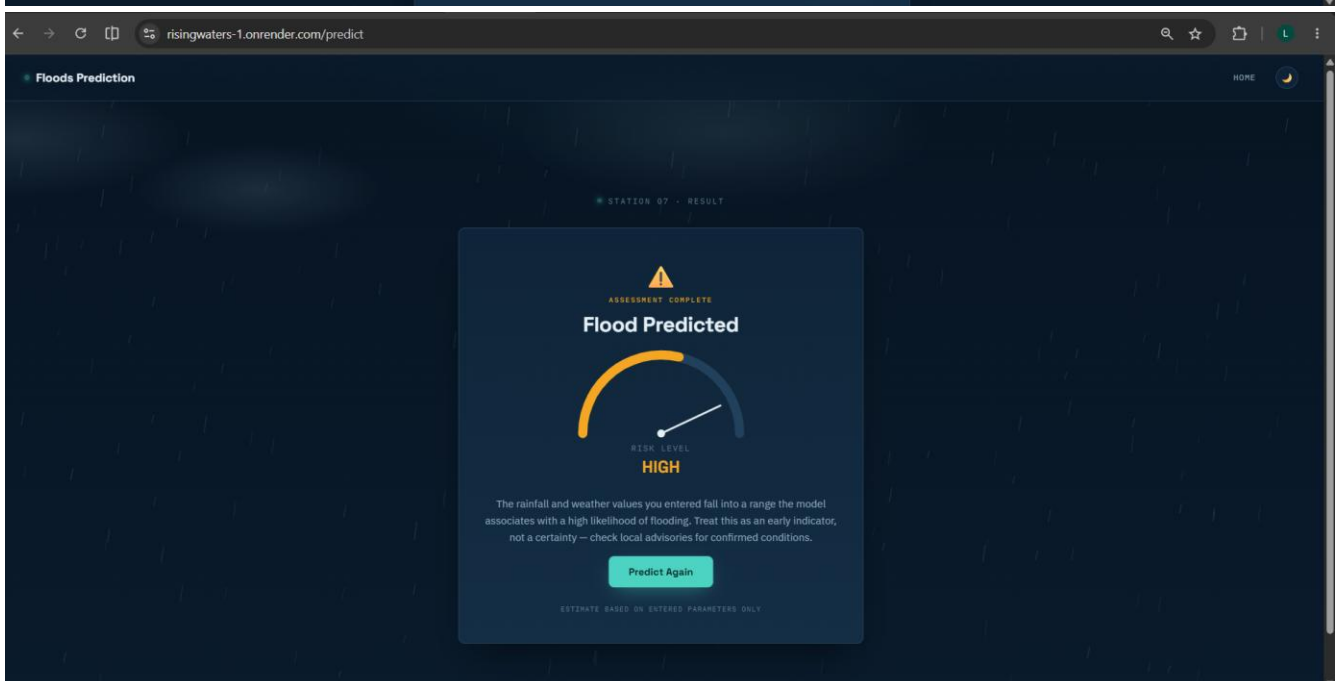
MARCH-MAY RAINFALL (MM) JUNE-SEPTEMBER RAINFALL (MM)

OCT-DEC RAINFALL (MM) AVERAGE JUNE RAINFALL (MM)

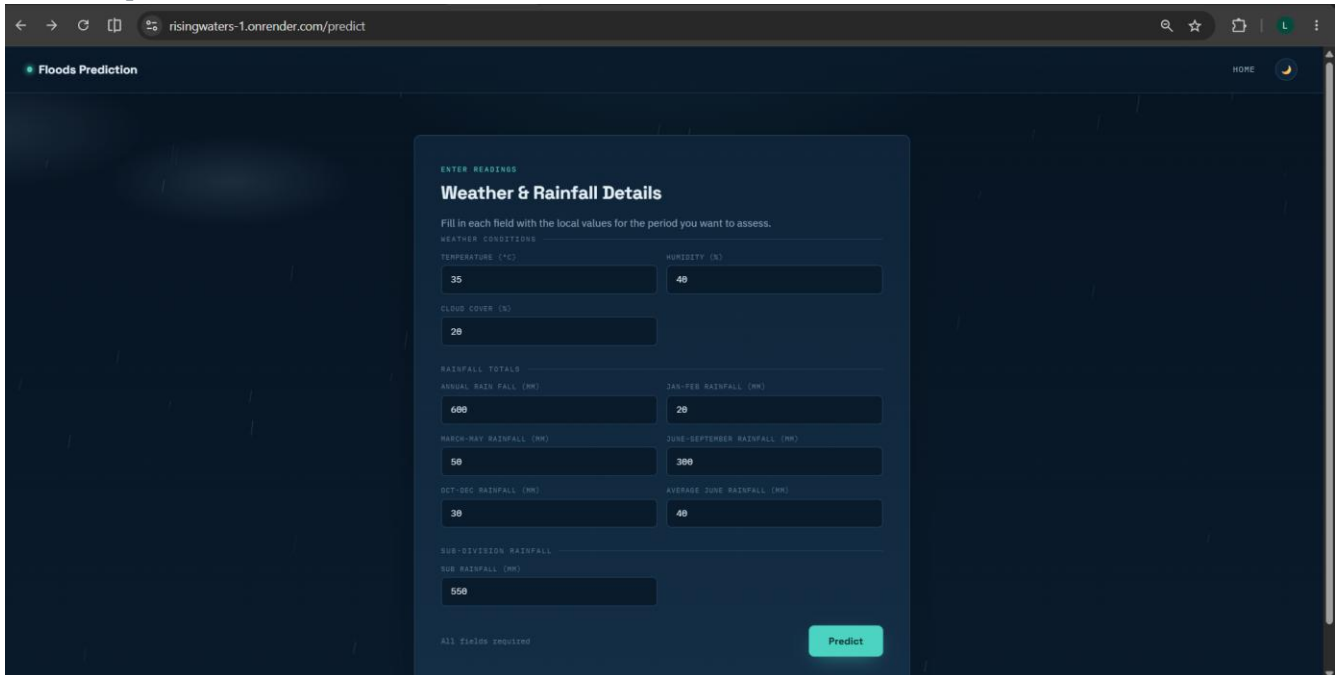
SUB-DIVISION RAINFALL

SUB RAINFALL (MM)

All fields required



- no_chance.html — shown when no flood is predicted; displays an animated low-risk gauge. Both link back to the prediction form.



Floods Prediction

ENTER READINGS

Weather & Rainfall Details

Fill in each field with the local values for the period you want to assess.

WEATHER CONDITIONS

TEMPERATURE (°C) HUMIDITY (%)

CLOUD COVER (%)

RAINFALL TOTALS

ANNUAL RAIN FALL (MM) JAN-FEB RAINFALL (MM)

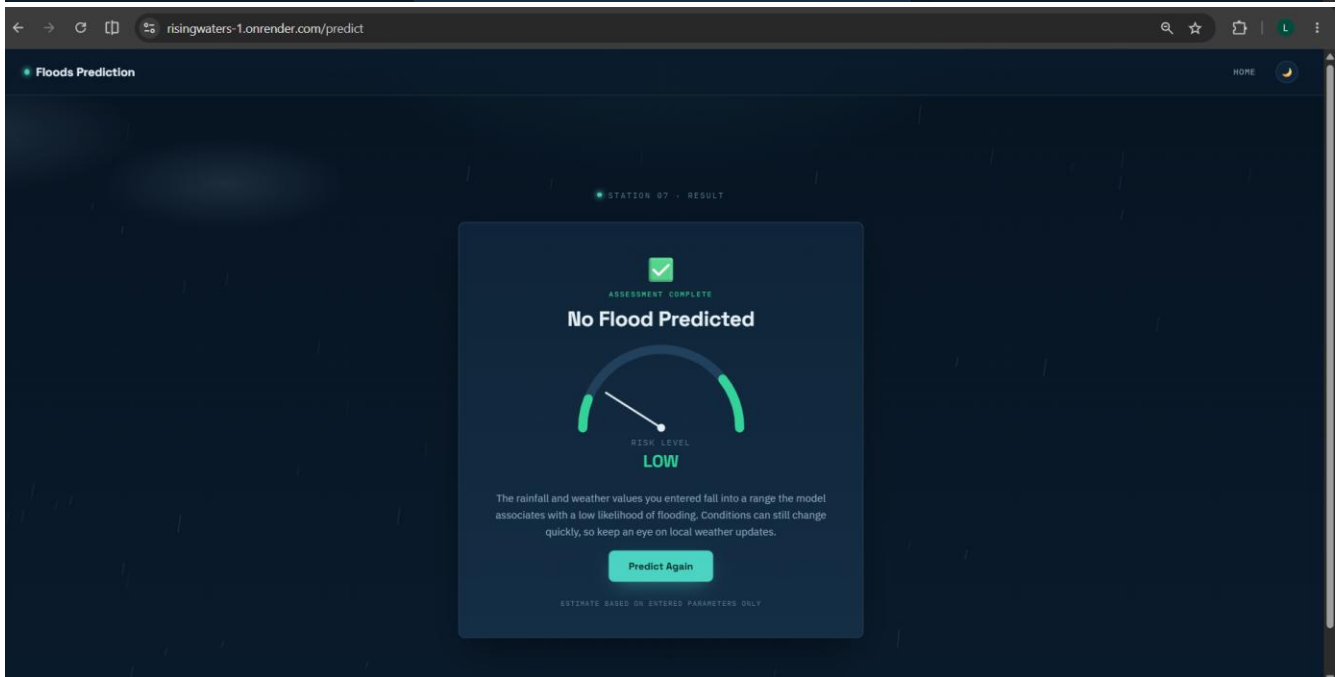
MARCH-MAY RAINFALL (MM) JUNE-SEPTEMBER RAINFALL (MM)

OCT-DEC RAINFALL (MM) AVERAGE JUNE RAINFALL (MM)

SUB-DIVISION RAINFALL

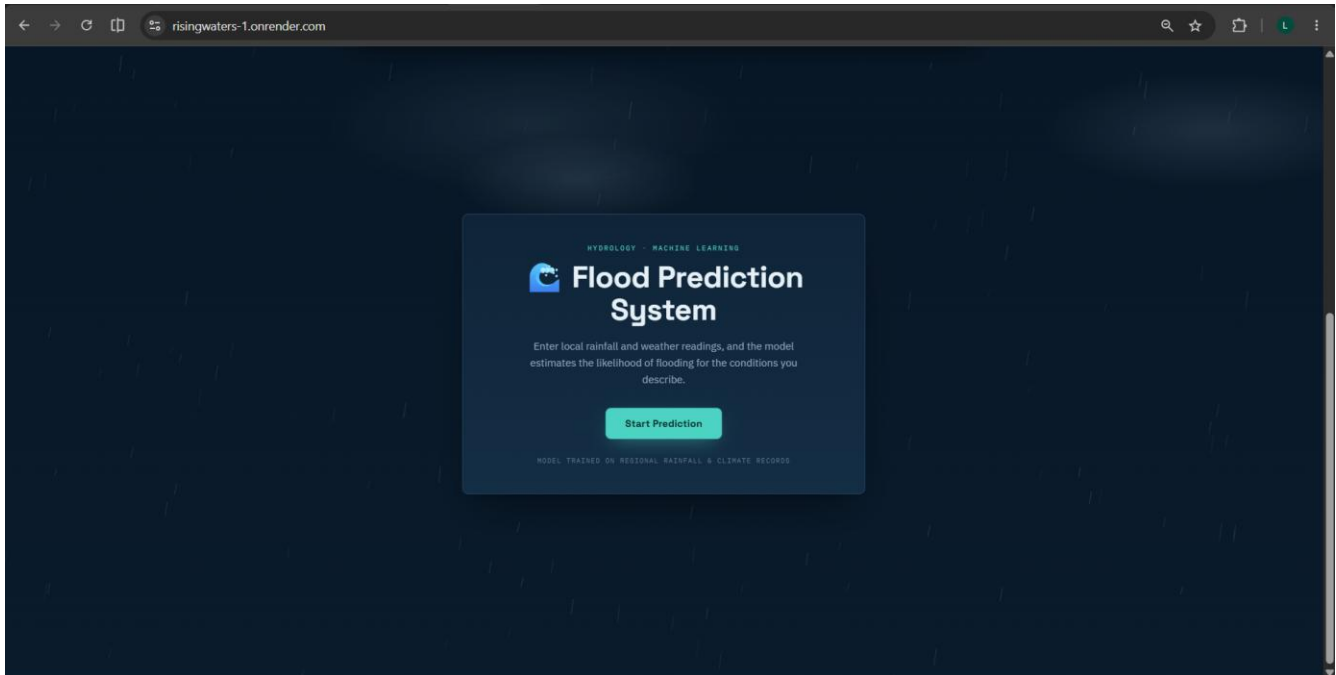
SUB RAINFALL (MM)

All Fields Required



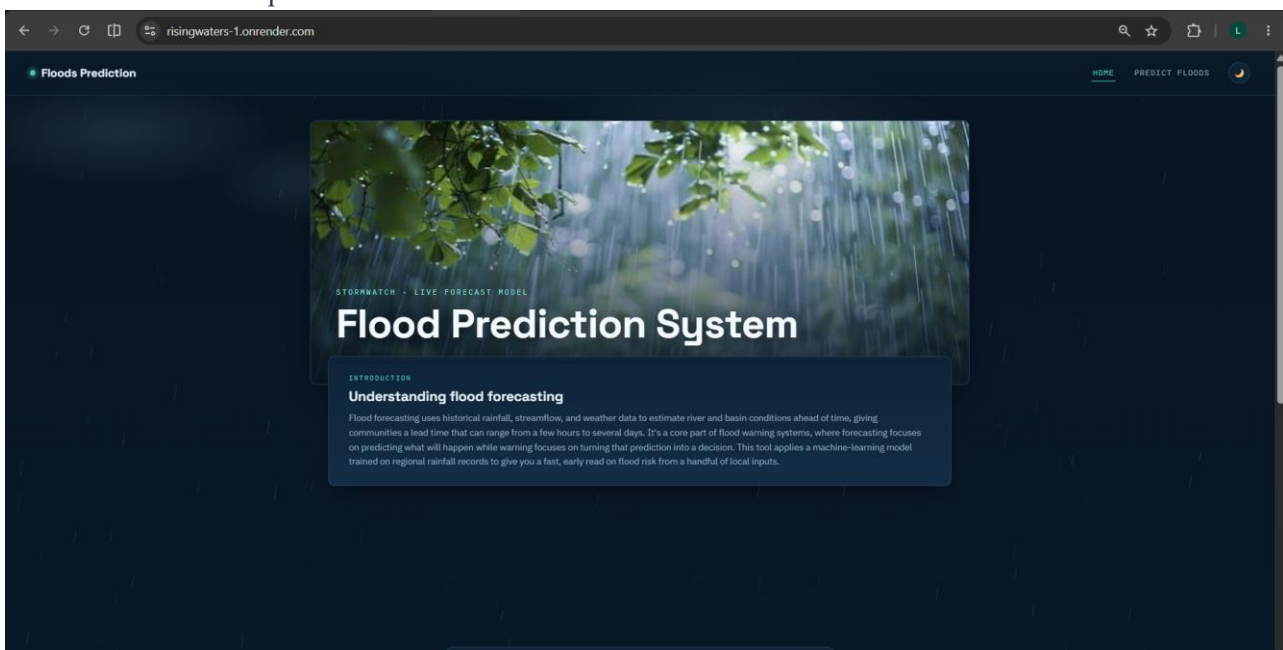
Activity 4.4: Ambient Canvas Animation

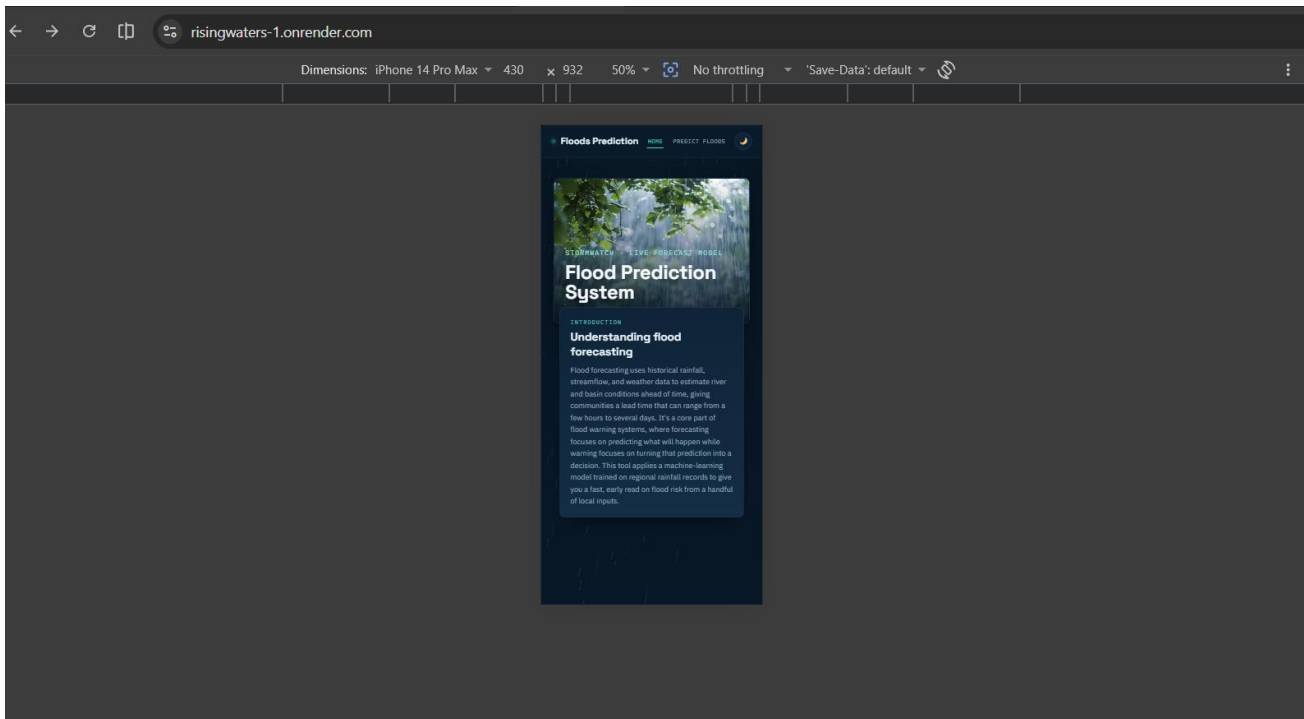
- Add a rain-and-lightning canvas animation in static/main.js for atmosphere across all pages.



Activity 4.5: Styling

- Implement the responsive layout and theme variables in static/main.css, ensuring the app adapts across mobile and desktop.





Milestone 5: Deployment (Render)

Activity 5.1: Prepare requirements.txt

```
Flask
gunicorn
NumPy
Pandas
Matplotlib
Seaborn
Scikit-learn
Joblib
```

Activity 5.2: Add a Procfile

Render's web service reads a Procfile to know how to start the app in production, using Gunicorn instead of Flask's development server:

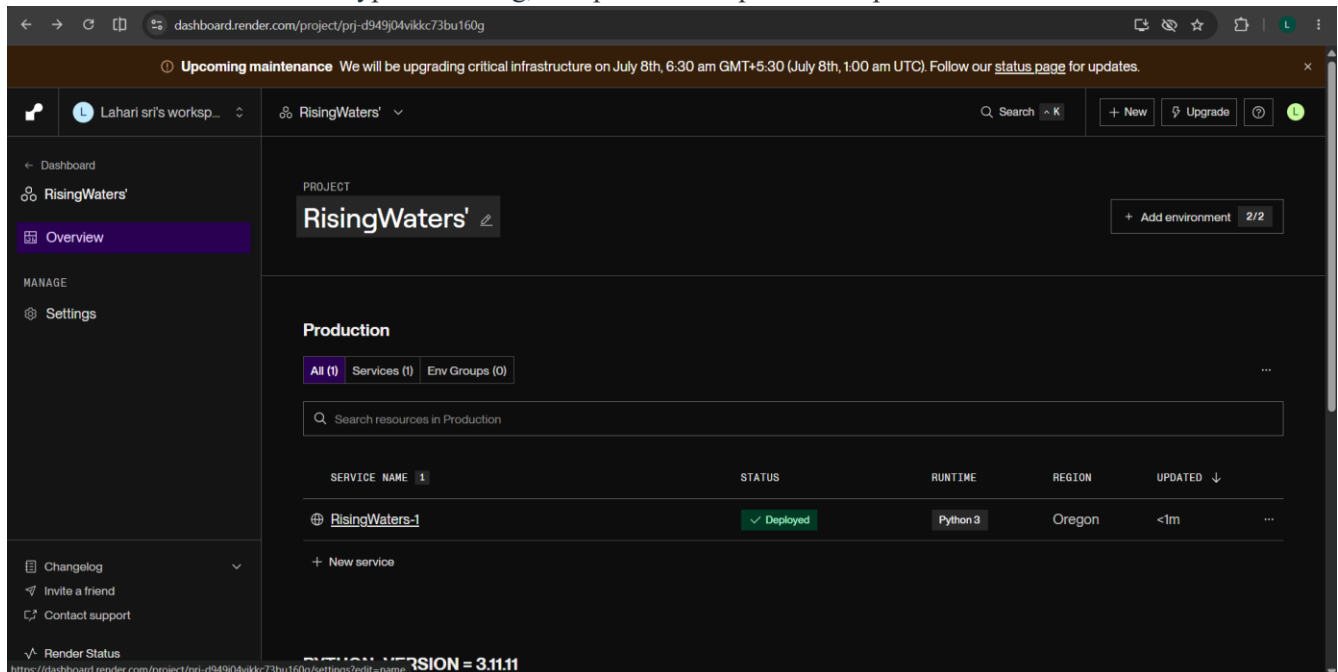
```
web: gunicorn app:app --bind 0.0.0.0:$PORT
```

Activity 5.3: Push the Repository to GitHub

```
git init
git add .
git commit -m "Initial commit - Rising Waters"
git remote add origin https://github.com/Laharisrikotipalli/RisingWaters.git
git push -u origin main
```

Activity 5.4: Create a Render Web Service

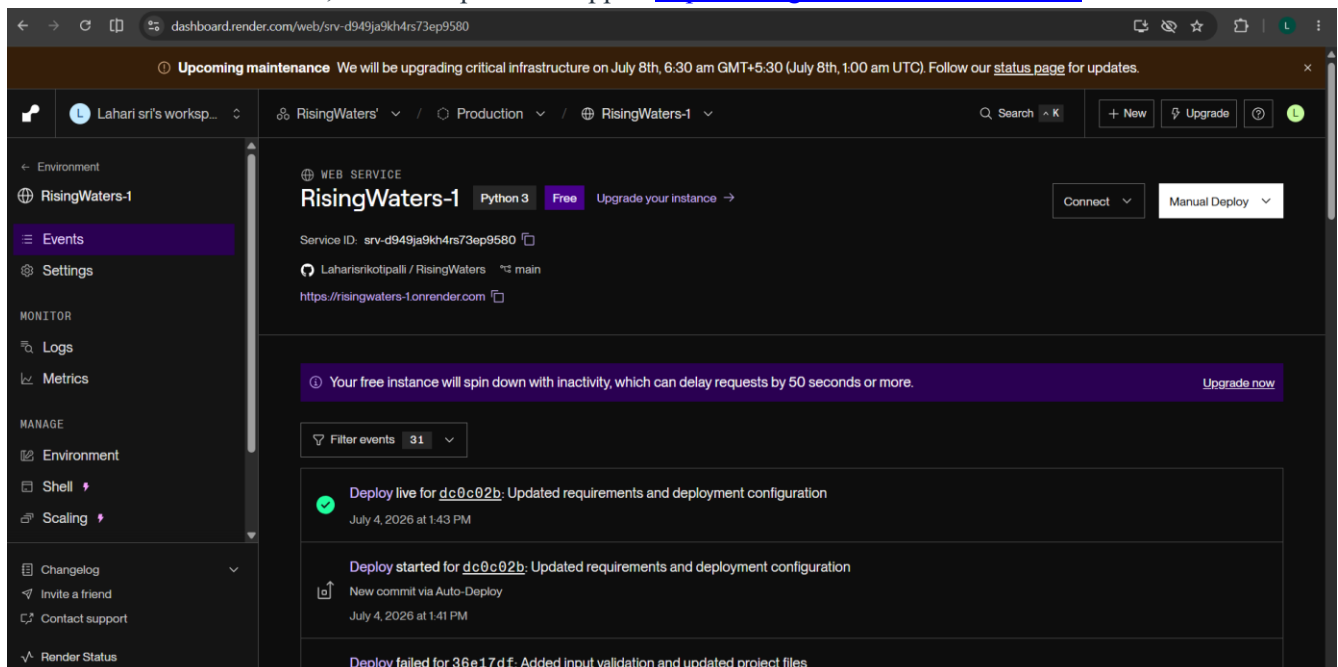
- Sign in to Render and choose New → Web Service.
- Connect the GitHub repository containing the Rising Waters project.
- Set the Build Command to `pip install -r requirements.txt`.
- Set the Start Command to `gunicorn app:app --bind 0.0.0.0:$PORT` (or leave it to the Procfile).
- Choose the free instance type for testing, or a paid tier for persistent uptime.



The screenshot shows the Render dashboard for a project named 'RisingWaters'. The left sidebar contains navigation links: Dashboard, Overview (selected), Settings, Changelog, Invite a friend, and Contact support. The main content area shows the 'Production' environment with a table of services. One service, 'RisingWaters-1', is listed with a status of 'Deployed', runtime of 'Python 3', region of 'Oregon', and updated less than 1 minute ago. A '+ New service' button is visible below the table. At the bottom, the 'PYTHON VERSION = 3.11.11' is noted.

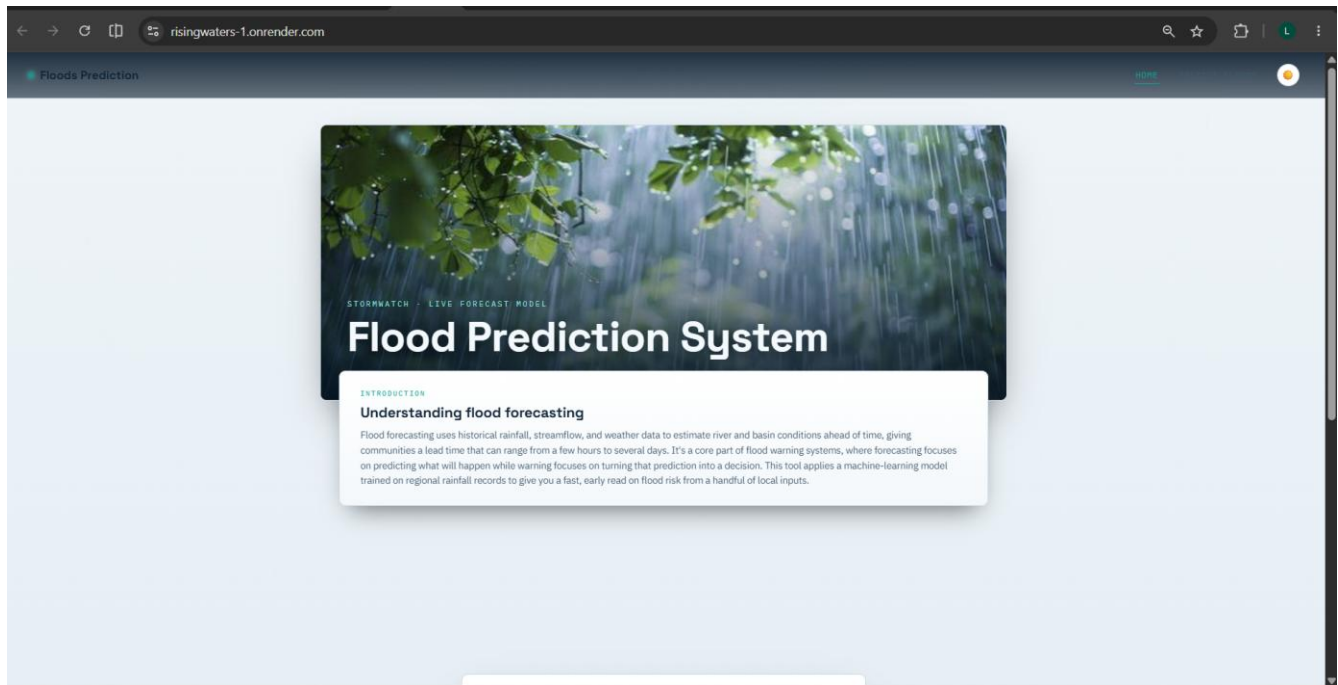
Activity 5.5: Verify the Live Deployment

- Once the build finishes, Render exposes the app at <https://risingwaters-1.onrender.com/>.



The screenshot shows the 'RisingWaters-1' service details page. The left sidebar has links for Environment, Events (selected), Settings, Logs, Metrics, Environment, Shell, and Scaling. The main content area shows the service is a 'WEB SERVICE' using 'Python 3' on a 'Free' instance. It includes a 'Connect' button and a 'Manual Deploy' button. A warning banner states: 'Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.' Below this, a list of deployment events is shown, including a successful deployment for 'dc0c02b' at 1:43 PM and a failed deployment for '36e17df'.

- Open the URL and test both prediction outcomes (Flood Predicted and No Flood Predicted) on the live instance.



Important — On Render's free tier, an inactive service spins down and takes a few seconds to wake up on the next request; this is expected behavior, not an error.

- ```
lahari@ASUS-VivoBook MINGW64 ~/OneDrive/Desktop/Rising-Waters (main)
$ curl http://127.0.0.1:5000/predict
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Flood Prediction</title>
 <link rel="preconnect" href="https://fonts.googleapis.com">
 <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;700&family=IBM+Plex+Sans:wght@400;500&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
 <link rel="stylesheet" href="/static/main.css">
</head>
<body data-rain="light">

 <canvas id="rain-canvas"></canvas>
 <div class="cloud-layer">
 <div class="cloud c1"></div>
 <div class="cloud c2"></div>
 </div>
 <div class="lightning-flash"></div>

 <nav class="navbar">
 <div class="nav-brand">Floods Prediction</div>
 <div class="nav-links">
 Home
 <button class="theme-toggle" id="theme-toggle" type="button" aria-label="Toggle light and dark theme">
 ☀
 </button>
 </div>
 </nav>

 <div class="page">

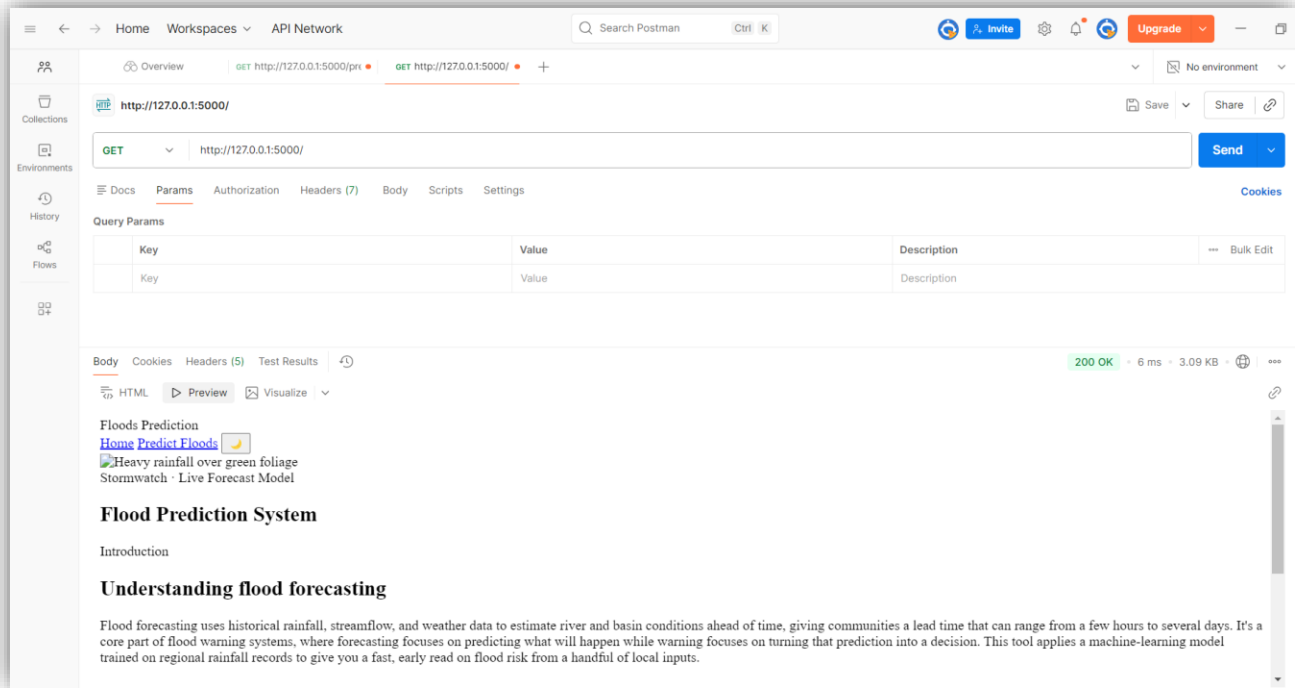
 <div class="panel wide">
 Enter Readings
 <h2 class="title-hero" style="font-size:28px;">Weather & Rainfall Details</h2>
 <p style="margin-bottom:0;">✖Fill in each field with the local values for the period you want to assess.</p>

 <form action="/predict" method="POST">

 <div class="section-label">Weather conditions</div>
 <div class="field-grid">
 <div class="field">
 <label for="Temp">Temperature (°C)</label>
 <input type="number" step="any" id="Temp" name="Temp" placeholder="Average Temperature" required>
 </div>
 <div class="field">
 <label for="Humidity">Humidity (%)</label>

```

- Activity 6.3: Document the setup, architecture, and workflow, and record a short demo walkthrough of the deployed app.



Postman interface showing a GET request to `http://127.0.0.1:5000/` with a 200 OK response. The response body contains the following HTML content:

```

Floods Prediction
Home Predict Floods
 Heavy rainfall over green foliage
Stormwatch - Live Forecast Model

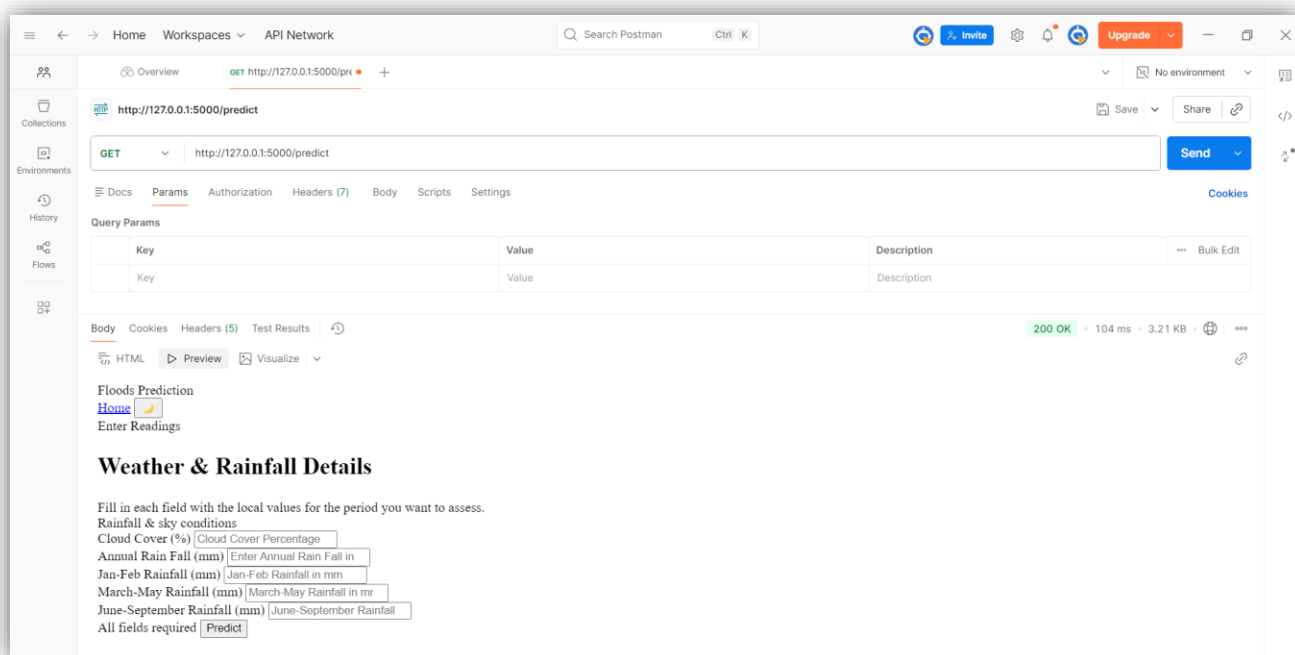
Flood Prediction System

Introduction

Understanding flood forecasting

Flood forecasting uses historical rainfall, streamflow, and weather data to estimate river and basin conditions ahead of time, giving communities a lead time that can range from a few hours to several days. It's a core part of flood warning systems, where forecasting focuses on predicting what will happen while warning focuses on turning that prediction into a decision. This tool applies a machine-learning model trained on regional rainfall records to give you a fast, early read on flood risk from a handful of local inputs.

```



Postman interface showing a GET request to `http://127.0.0.1:5000/predict` with a 200 OK response. The response body contains the following HTML content:

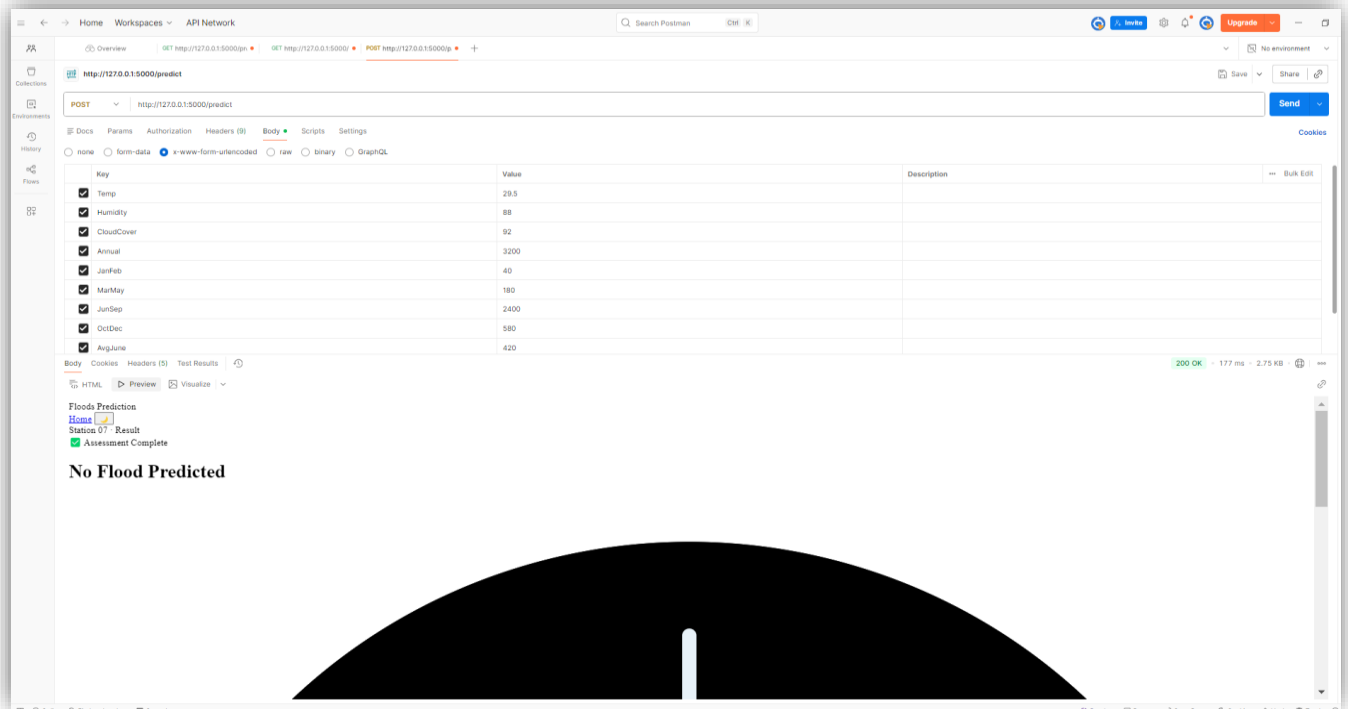
```

Floods Prediction
Home
Enter Readings

Weather & Rainfall Details

Fill in each field with the local values for the period you want to assess.
Rainfall & sky conditions
Cloud Cover (%)
Annual Rain Fall (mm)
Jan-Feb Rainfall (mm)
March-May Rainfall (mm)
June-September Rainfall (mm)
All fields required

```



The screenshot shows the Postman API client interface. The top bar indicates the current workspace is 'API Network'. The main area displays a POST request to the endpoint `http://127.0.0.1:5000/predict`. The request body is a JSON object with the following data:

| Key        | Value | Description |
|------------|-------|-------------|
| Temp       | 29.5  |             |
| Humidity   | 88    |             |
| CloudCover | 92    |             |
| Annual     | 3200  |             |
| JanFeb     | 40    |             |
| MarMay     | 180   |             |
| Jun-Sep    | 2400  |             |
| OctDec     | 580   |             |
| AvgJune    | 420   |             |

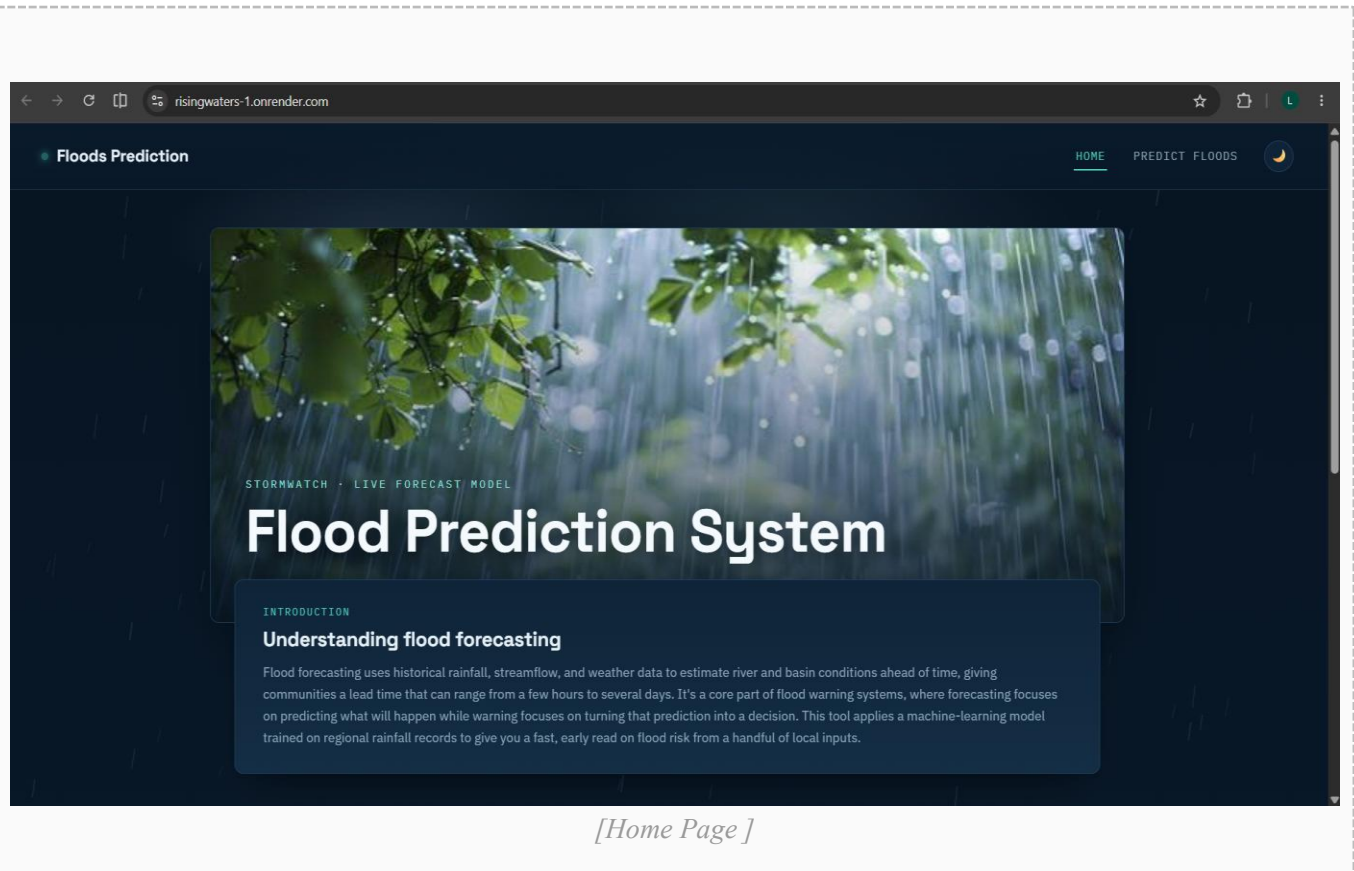
The response status is **200 OK** with a response time of 177 ms and a body size of 2.75 KB. The response body is HTML, showing a 'Floods Prediction' result for 'Station 07'. The prediction is 'No Flood Predicted'.

**Floods Prediction**  
Station 07 Result  
Assessment Complete  
**No Flood Predicted**

## Exploring the Website's Web Pages

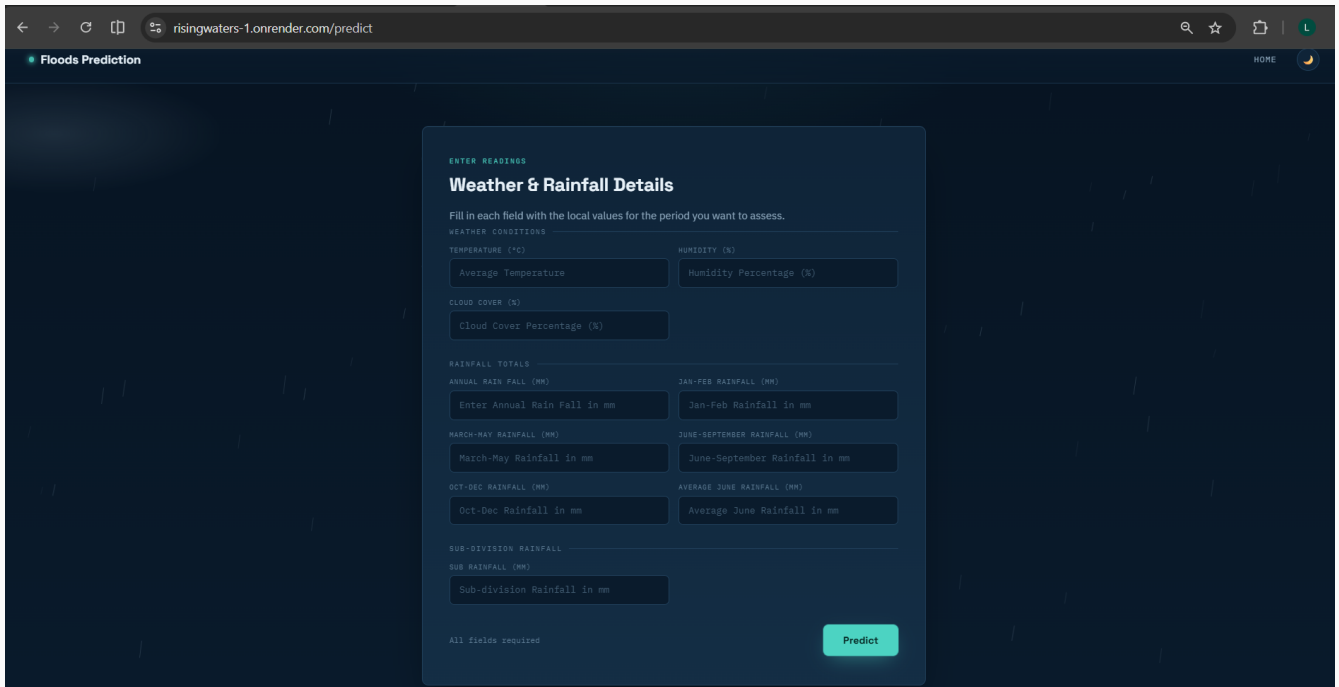
### Home Page

Introduces the project with a hero section and background on flood forecasting, and a call to action linking to the prediction form.



## Prediction Page

A form collecting all 10 model inputs, grouped into Weather Conditions, Rainfall Totals, and Sub-division Rainfall sections, with a light/dark theme toggle available at all times.



The screenshot shows a web browser window with the URL `risingwaters-1.onrender.com/predict`. The page title is "Floods Prediction". The form is titled "Weather & Rainfall Details" and includes the instruction: "Fill in each field with the local values for the period you want to assess."

**WEATHER CONDITIONS**

- TEMPERATURE (°C): Average Temperature
- HUMIDITY (%): Humidity Percentage (%)
- CLOUD COVER (%): Cloud Cover Percentage (%)

**RAINFALL TOTALS**

- ANNUAL RAIN FALL (MM): Enter Annual Rain Fall in mm
- JAN-FEB RAINFALL (MM): Jan-Feb Rainfall in mm
- MARCH-MAY RAINFALL (MM): March-May Rainfall in mm
- JUNE-SEPTEMBER RAINFALL (MM): June-September Rainfall in mm
- OCT-DEC RAINFALL (MM): Oct-Dec Rainfall in mm
- AVERAGE JUNE RAINFALL (MM): Average June Rainfall in mm

**SUB-DIVISION RAINFALL**

- SUB RAINFALL (MM): Sub-division Rainfall in mm

All fields required

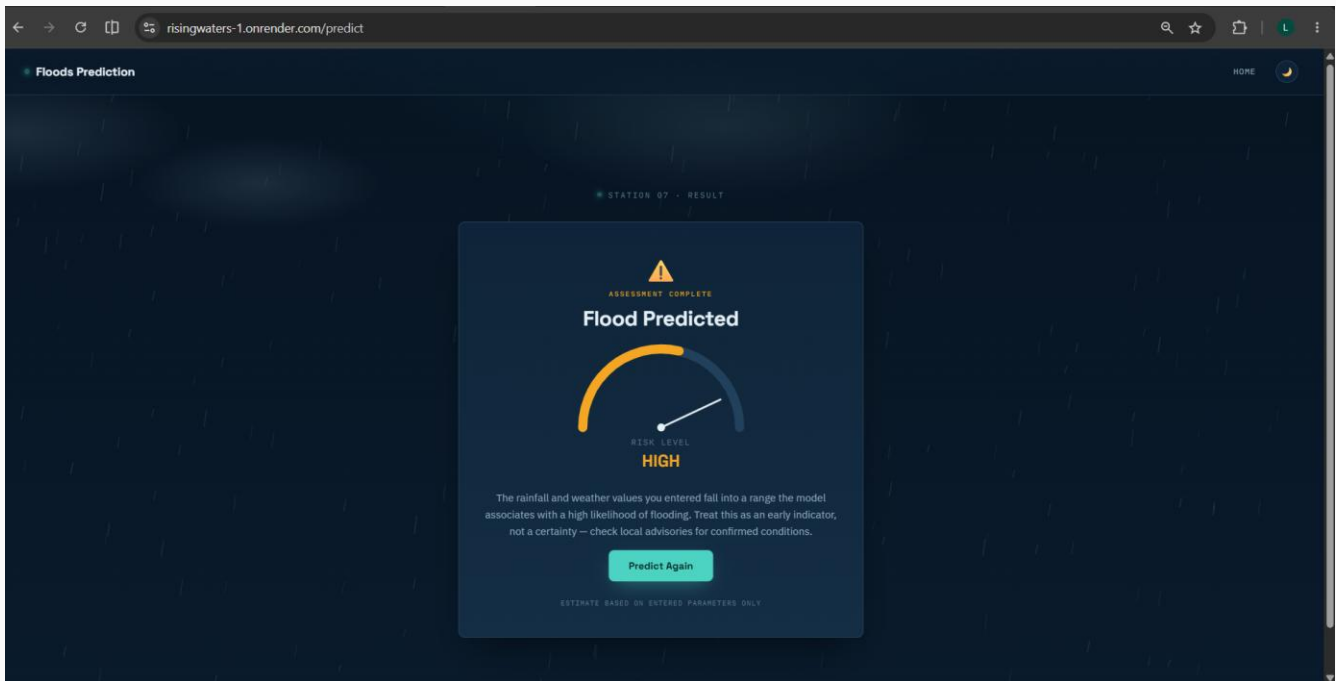
**Predict**

[ Prediction Page ]

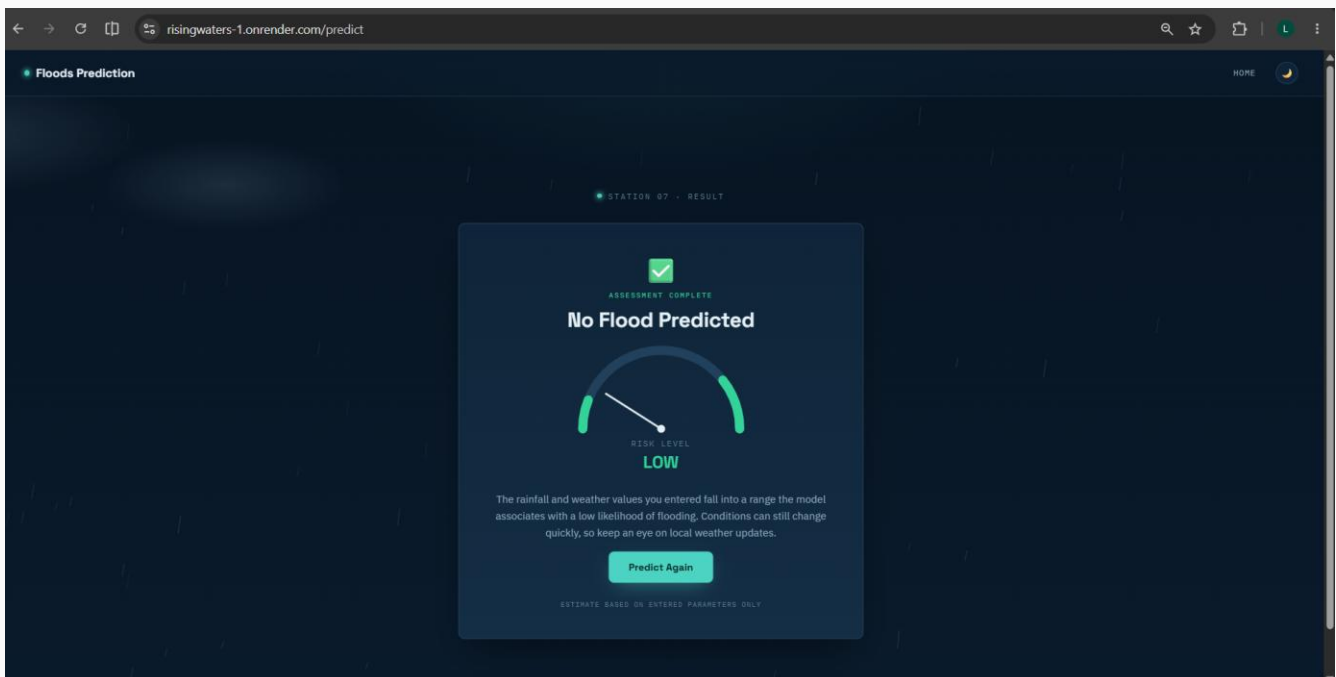
## Result Pages

chance.html shows a Flood Predicted message with a high-risk animated gauge; no\_chance.html shows a No Flood Predicted message with a low-risk gauge. Both pages link back to the prediction form for a new reading.





*Result Page (Flood Predicted)*



*( No Flood Predicted)*

## Conclusion

Rising Waters is a machine learning platform built with Flask that predicts flood occurrence from ten weather and rainfall readings. It compares classical classification models, serializes the best-performing pipeline with Joblib, and serves predictions instantly through a responsive, themeable web interface complete with an animated risk gauge and ambient rain effects. The project, which spanned data preparation, model training, Flask development, and deployment to Render, demonstrates how a compact ML pipeline can be packaged into a usable public tool. Looking ahead, Rising Waters has clear room to grow real-time weather API integration, river and dam water-level monitoring, satellite data, deep learning-based forecasting, and a mobile companion app are all natural next steps toward a more comprehensive flood early-warning system.